

PCBNN: An Algorithm–Hardware Co-Designed Binary Neural Network for Low-Power Edge Inference

Haihui Liu, Baoxu An, Yaxin Mei*, Yunxuan Zhu, Guangxue Zhang, Yuzhu Liang

Abstract Deploying deep neural networks on resource-constrained edge computing systems requires high inference efficiency, low power consumption, and compact model representation, while maintaining competitive accuracy on large-scale tasks. To address these challenges, this paper proposes PCBNN, an algorithm–hardware co-designed binary neural network for low-power edge inference on FPGAs. PCBNN introduces a parallel binary convolutional architecture that exploits fine-grained hardware parallelism to accelerate inference. On the algorithmic side, a contrastive learning framework is designed to explicitly model the correlation between binary activations and their full-precision counterparts, thereby mitigating accuracy degradation from binarization. Furthermore, a channel-segmentation scheme enables a fully binarized input layer, preserving critical features while reducing computational complexity. Building on these components, an FPGA-based accelerator is implemented to support efficient and low-latency BNN inference. The experimental results demonstrate that PCBNN effectively utilizes FPGA parallelism and hardware resources to achieve substantial improvements in both accuracy and energy efficiency compared with state-of-the-art BNN accelerators.

Keywords Binary neural networks, contrastive learning, edge computing, FPGAs, hardware accelerators

1 Introduction

With the rapid proliferation of intelligent sensing applications, massive perceptual data are increasingly generated at the network edge, which imposes stringent requirements on real-time and energy-efficient processing in resource-constrained edge computing systems [1–4]. Recent studies on mobile edge intelligence, federated digital twins, IoT data dissemination, and green data gathering further indicate that edge systems increasingly involve distributed sensing, adaptive model update, resource allocation, and energy-constrained data processing [5–10]. In practical scenarios such as intelligent surveillance [11], industrial inspection [12], AIoT sensory data processing [13], and mobile vision [14], it is often desirable to perform

inference close to data sources in order to reduce response latency and alleviate communication overhead. Deep neural networks (DNNs) [15], especially convolutional neural networks (CNNs) [16], have achieved remarkable success in perception and recognition tasks. However, their increasing model size, memory footprint, and computational complexity make direct deployment on edge devices increasingly difficult [17]. Therefore, enabling accurate and low-power neural inference under tight hardware budgets has become a critical challenge for edge intelligence systems [18, 19].

To address the deployment bottleneck of deep models at the edge, various model compression techniques have been developed, including network pruning [20], knowledge distillation [21], and network quantization [22]. Among them, network binarization [23] is particularly attractive for edge computing because it constrains both weights and activations to 1-bit representations, thereby significantly reducing model size and computational cost. Courbariaux *et al.* [24] first introduced binary neural networks (BNNs), in which most floating-point multiply-and-accumulate operations are replaced by XNOR and Popcount operations. Prior studies have shown that 1-bit convolutional layers can achieve up to $32\times$ memory reduction and $58\times$ CPU speedup compared with full-precision counterparts [23]. Owing to their compact model representation and bitwise-friendly computation, BNNs have become a promising solution for low-power edge computing systems [25].

From the hardware perspective, field-programmable gate arrays (FPGAs) [26, 27] provide an appealing deployment substrate for BNNs because of their reconfigurability, fine-grained parallelism, and favorable energy efficiency. By mapping binary multiply-and-accumulate operations to bitwise logic, FPGA-based accelerators can exploit massive on-chip parallelism while reducing the cost of floating-point computation. Consequently, a number of studies [28–31] have investigated BNN deployment on FPGA platforms, aiming to improve throughput and hardware utilization while reducing on-chip memory consumption and off-chip data movement [32–34].

Despite these advantages, the practical deployment of BNNs in edge computing systems is still hindered by two

Haihui Liu, Baoxu An, Yaxin Mei, Yunxuan Zhu, Guangxue Zhang and Yuzhu Liang are with the Institute of Artificial Intelligence and Future Networks, Beijing Normal University, Zhuhai 519087, China. E-mail: haihui1iu@mail.bnu.edu.cn; anbaoxu@mail.bnu.edu.cn; yxmei@mail.bnu.edu.cn; 202411109033@mail.bnu.edu.cn; guangxue_zhang@bnu.edu.cn; yuzhuliang

major challenges [35]. First, BNNs often suffer from considerable accuracy degradation compared with full-precision networks, especially on large-scale datasets and complex visual tasks. Although existing studies have improved binary model performance [36, 37], the accuracy gap on challenging benchmarks such as ImageNet remains non-negligible. Second, the computational benefits of binarization are not always fully translated into system-level efficiency on edge hardware. In many existing designs, the input layer still relies on fixed-point computation, and the mismatch between algorithm design and hardware architecture may introduce additional resource overhead, increased data movement, and reduced deployment efficiency [35]. As a result, improving only model accuracy or only hardware throughput is insufficient. A more effective solution requires algorithm-hardware co-design that jointly considers representation capability, binary computability, and hardware deployability.

Motivated by this observation, this paper investigates the co-design of BNN architecture and FPGA accelerator for efficient large-scale edge inference. Our objective is to jointly improve classification accuracy, throughput, and energy efficiency under the resource constraints of practical edge platforms. To this end, we propose PCBNN, a parallel-convolution and contrastive-learning optimized binary neural network for FPGA-based low-power edge inference. Unlike conventional methods that separately optimize binary model design or hardware implementation, PCBNN is developed from a unified co-design perspective to improve accuracy-efficiency-deployability simultaneously. The main contributions of this paper are summarized as follows:

- We propose an algorithm-hardware co-designed BNN framework, termed PCBNN, for low-power edge computing systems, which jointly considers model accuracy, hardware efficiency, and deployability on FPGA-based edge platforms.
- We develop a parallel convolutional architecture and a contrastive learning mechanism for PCBNN to improve the representation capability of binary networks and mitigate information loss caused by activation binarization.
- We introduce a channel segmentation method to realize a fully binarized input layer, which helps preserve informative input features while reducing the computational complexity and hardware overhead of subsequent processing.
- We design and implement an FPGA-based PCBNN accelerator using high-level synthesis, enabling efficient, low-latency, and energy-efficient BNN inference for resource-constrained edge deployment.

The remainder of this paper is organized as follows. Section 2 reviews related work on BNNs and FPGA-based accelerators. Section 3 presents the proposed PCBNN model. Section 4 describes the FPGA-based implementation and optimization of PCBNN. Section 5 reports the experimental results and performance analysis. Finally, Section 6 concludes this paper.

2 Related work

Existing studies on BNNs and FPGA-based accelerators have improved binary representation accuracy and hardware efficiency from different perspectives. However, these two directions are often optimized separately, and many accelerators still rely on fixed-point or low-bit input-layer processing, which weakens the benefit of full binarization. Therefore, a unified design that jointly considers binary representation, input-layer binarization, and FPGA-friendly computation is still needed. PCBNN addresses this gap through a parallel binary convolutional architecture, contrastive-learning-guided activation optimization, a channel-segmentation-based binary input layer, and a corresponding FPGA accelerator.

2.1 Binary neural networks

Binary neural networks (BNNs) were first introduced by Courbariaux *et al.* [24], where both network weights and activations are constrained to binary values to significantly reduce model size and arithmetic complexity. Subsequent studies and implementations [38] verified the feasibility of BNNs for compact and efficient inference, especially on small-scale datasets such as MNIST and CIFAR-10.

Despite their efficiency advantages, BNNs generally suffer from severe accuracy degradation due to aggressive quantization. To alleviate quantization errors and improve model performance, a number of methods have been proposed from different perspectives. Lin *et al.* [39] introduced Rotation BNN to reduce the impact of angular bias during quantization. Shang *et al.* [40] improved binary quantization by exploiting activation distribution characteristics and the intrinsic properties of BNNs. Tu *et al.* [41] further proposed a layer-wise adaptive strategy to obtain more suitable binary weights and activations, narrowing the performance gap between binary and full-precision networks.

More recently, BNN research has continued to advance toward higher recognition accuracy. Vargas *et al.* [42] proposed BiPer, which uses a periodic binarization function and its differentiable surrogate to better control quantization error during training. Ma *et al.* [43] developed A&B BNN, a hardware-friendly binary network that replaces multiplication-unfriendly operations with add and bit op-

erations while maintaining competitive classification performance. Orecchia *et al.* [44] proposed a new BNN training framework based on Gaussian variational inference and low-rank semidefinite programming, and reported strong results on CIFAR-10, CIFAR-100, Tiny-ImageNet, and ImageNet. Although these studies have substantially improved BNN accuracy, jointly achieving strong recognition performance, strict binary computation, and efficient edge deployment remains challenging.

Unlike methods that mainly improve training strategies or binary quantizers, PCBNN further considers hardware deployability. Its parallel convolutional block enhances binary feature representation while exposing regular parallelism for FPGA implementation, thereby jointly improving representation capability and execution efficiency.

2.2 Quantization of input layer

Quantizing the input layer is essential for realizing fully binary computation. However, in many existing BNN models, the first layer is still processed in fixed-point format. For example, Zhao *et al.* [34] and Nakahara *et al.* [45] designed separate fixed-point processing modules for the input layer, while Nurvitadhi *et al.* [46] proposed a shareable structure that can be configured either as 32 1-bit multipliers or as full-precision multipliers. Although these methods support efficient first-layer processing, the extra fixed-point hardware reduces resource efficiency and weakens the overall benefits of binarization.

To address this issue more fundamentally, several studies have explored direct binarization of the input layer. Hirtzlin *et al.* [47] binarized input images using stochastic computing, expanding the three RGB channels to more than 1500 binary channels. While this approach simplifies arithmetic operations, it also increases the number of parameters and MAC operations by nearly $16\times$. Guo *et al.* [48] represented each pixel as a sum of binary vectors so that all convolution operations in the first layer can be implemented in binary form. However, the vector dimension must be a power of two, and large pixel values may cause exponential accuracy degradation. Zhang *et al.* [49] adopted thermometer encoding to binarize the input layer while preserving the advantages of BMAC, but their method still requires additional sparse binary convolutional layers and incurs non-negligible resource overhead for large-scale models.

Overall, existing studies reveal that input-layer binarization is closely related to both model accuracy and hardware efficiency. How to balance binary input representation, feature

preservation, and implementation overhead remains an open problem for low-power edge computing systems.

Compared with prior input-layer binarization methods, the proposed channel segmentation scheme avoids a dedicated fixed-point first-layer engine. It converts each RGB input channel into multiple binary channels, enabling the first layer to share the same XNOR-and-Popcount pipeline as subsequent layers and reducing hardware complexity.

2.3 FPGA-based accelerator for BNNs

FPGAs are well suited for implementing BNNs because of their strong computational capability, fine-grained parallelism, and high flexibility [32, 34, 48–52]. By mapping binary operations to lightweight logic such as XNOR and Popcount, FPGA-based accelerators can efficiently exploit on-chip parallelism and reduce the cost of conventional floating-point computation. Early studies demonstrated the practicality of BNN acceleration on FPGAs. Zhao *et al.* [34] were among the first to implement BNN accelerators on FPGA platforms and introduced edge-filling retraining techniques to improve inference efficiency. Liang *et al.* [32] proposed a resource-aware model analysis method that uses XNOR and Popcount operations to alleviate the multiplier bottleneck and improve inference performance.

To further improve both efficiency and accuracy, later studies investigated more advanced binary accelerator designs. Guo *et al.* [48] proposed FBNA, a fully binarized CNN accelerator that employs an even-odd padding scheme to improve resource utilization and recognition performance on small datasets. Ghasemzadeh *et al.* [50] developed ReBNet, which combines residual binarization with a reconfigurable FPGA accelerator to enable a tunable trade-off between accuracy and hardware cost. Zhang *et al.* [49] proposed FracBNN, which supports fractional activations and a thermometer-encoding-based input layer, achieving improved accuracy and frame rate while maintaining FPGA efficiency.

In addition to purely binary accelerators, several studies have explored low-bit or compressed CNN acceleration to better exploit FPGA resources. Nakahara *et al.* [51] integrated JPEG compression and convolutional processing into a unified FPGA pipeline, achieving high throughput while reducing input bandwidth and storage requirements. More recently, Zhang *et al.* [52] designed an FPGA-based BNN accelerator with enhanced hardware efficiency and data reuse, improving on-chip dataflow and reducing redundant memory access to achieve favorable throughput and power consumption.

Although these studies demonstrate the strong potential of FPGA-based accelerators for efficient neural inference, most

existing efforts emphasize either model-side optimization or hardware-side acceleration. There is still considerable room for algorithm-hardware co-design that jointly considers binary model representation, input-layer binarization, and efficient deployment on resource-constrained edge platforms.

In contrast to accelerator designs that optimize hardware dataflow after fixing the model, PCBNN adopts an algorithm-hardware co-design strategy. The model architecture, input representation, and FPGA pipeline are jointly designed so that binary convolution, activation binarization, and on-chip data reuse can be efficiently mapped to FPGA resources.

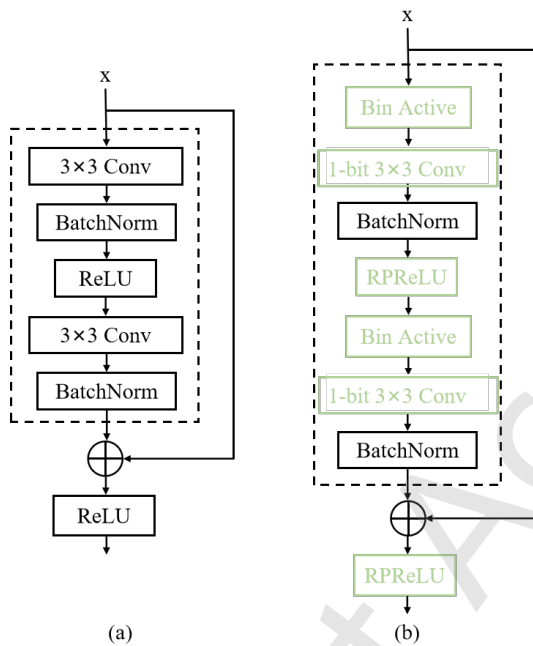


Fig. 1 Building blocks: (a) ResNet baseline building blocks; (b) Parallel convolutional building blocks in PCBNN.

3 PCBNN model

To address the above challenges, this section presents the proposed PCBNN model for efficient and accurate binary inference on FPGA-based edge platforms. We first describe the parallel convolutional architecture that enhances the representational capability and computational parallelism of binary networks. We then introduce a contrastive learning framework to alleviate information loss caused by activation binarization. Finally, we present the fully binarized input layer, which is designed to improve hardware efficiency and support low-latency deployment.

3.1 Parallel convolutional architecture for PCBNN

This subsection presents the parallel convolutional architecture of PCBNN. Since ResNet provides stable optimization

behavior and has been widely adopted in binary neural networks [40, 53], we use it as the baseline architecture and redesign its building block for more efficient binary inference.

Fig. 1(a) shows the original ResNet building block, while Fig. 1(b) illustrates the modified block used in PCBNN. Specifically, a Bin Active layer is inserted before the 3×3 convolution so that both input activations and weights can be processed in binary form through XNOR and Popcount operations. In addition, RPRReLU [37] is adopted to enhance the representation capability of the binary network, and shortcut connections are retained to stabilize optimization and facilitate feature propagation. Compared with the baseline ResNet block, PCBNN preserves the residual shortcut connection while adding Bin Active, 1-bit convolution, RPRReLU, and a binary-friendly computation path. These modifications enable the block to maintain stable feature propagation while making the main convolutional computation compatible with XNOR-and-Popcount-based binary inference.

Based on the above modifications, we further construct a parallel convolutional architecture, as shown in Fig. 2. In this design, the original binary convolution in the conventional block is replaced by two parallel binary convolution branches. Such a structure improves feature extraction capability while exposing more parallelism for hardware implementation, which is beneficial for both classification accuracy and efficient FPGA deployment. In other words, the dual-branch design extends the single binary convolution path into two parallel feature-extraction paths, with the primary purpose of enhancing the representation capability of binary features and providing more structured parallelism for the FPGA accelerator. The main components of the parallel convolutional block are described as follows.

- (1) **Bin Active:** Bin Active is used to generate binary activations, whose outputs take values in $\{+1, -1\}$. At the beginning of each stage, the input tensor is binarized by the sign function:

$$q = \text{Sign}(r) = \begin{cases} +1, & r \geq 0 \\ -1, & r < 0. \end{cases} \quad (1)$$

Here, q denotes the binarized result of input signal r . During forward propagation, the sign function performs activation binarization directly. For backward propagation, since the $\text{Sign}()$ function is not differentiable at the origin and the derivative is 0 at other coordinates, we use the straight-through estimator (STE) [54] strategy for gradient calculation, as shown in Eq. (2).

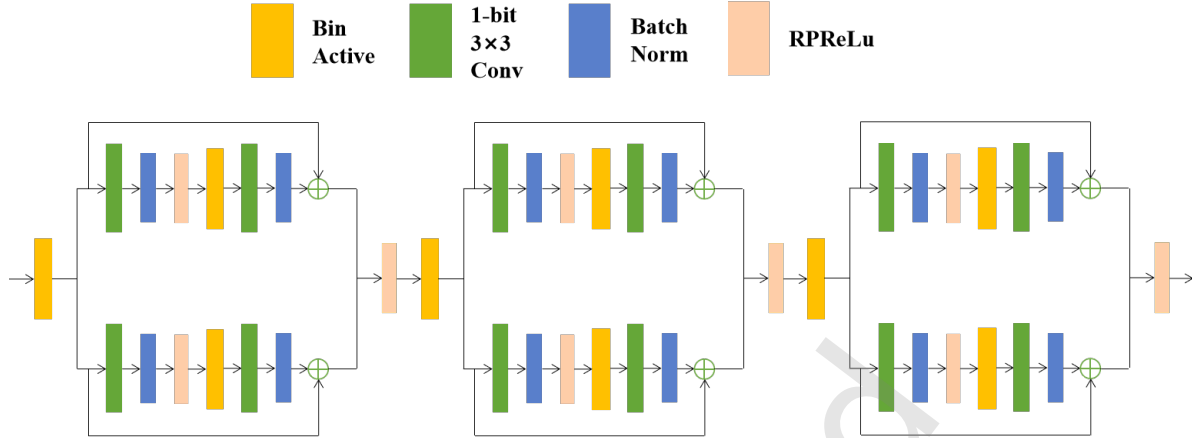


Fig. 2 Parallel convolutional building blocks of PCBNN. Each block contains two parallel binary convolution branches with Bin Active, 1-bit 3×3 convolution, BatchNorm, RPRelu, and residual shortcut connections.

$$\frac{\partial g}{\partial r} = \begin{cases} \frac{\partial g}{\partial q} & |r| \leq 1 \\ 0 & |r| > 1. \end{cases} \quad (2)$$

- (2) **Bin Conv:** Bin Conv refers to the convolutional operation on binary inputs and weights in BNNs, converting floating-point operations into XNOR and Popcount operations. This conversion is easy to map and parallelize on the look-up tables (LUTs) of FPGAs. Fig. 3 shows an example that has the binary convolution operation of 5×5 input tensors and 3×3 weights. The MAC operations are performed using XNOR and Popcount operations, completing the entire convolution process.

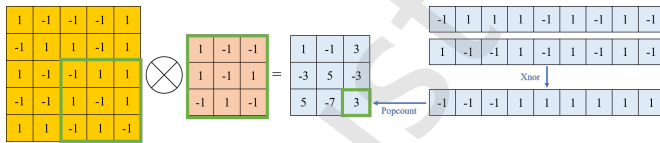


Fig. 3 Binary convolution by XNOR and Popcount.

- (3) **BatchNorm:** BatchNorm is applied after Bin Conv to normalize intermediate activations within each mini-batch, ensuring network stability, accelerating learning, and preventing overfitting. The mathematical expression of BatchNorm is:

$$y = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot \gamma + \beta. \quad (3)$$

The variance and mean of the batch data are denoted by σ^2 and μ , respectively. To avoid the denominator being 0, a small positive number is introduced, denoted as ϵ . The optimal learning parameters γ and β are obtained during training which are used to shift and scale the

normalized activation.

- (4) **RPRelu:** To improve the representation ability of the binary network, PCBNN adopts RPRelu [37] as the activation function between convolutional layers. RPRelu introduces learnable parameters to adaptively shift the activation distribution and reshape its response, which is helpful for reducing the information loss caused by binarization. Its formulation is expressed as

$$f(x) = \begin{cases} x - m + n, & x \geq m \\ k(x - m) + n, & x < m, \end{cases} \quad (4)$$

where m and n are learnable parameters that determine the transition point and output offset, and k controls the slope in the negative branch. Compared with conventional activations, RPRelu provides greater flexibility for feature transformation in binary networks and contributes to more accurate representation learning.

3.2 Contrastive learning framework for PCBNN

Quantization efficiently reduces storage and computational complexity, but often at the cost of accuracy. Inspired by the contrastive learning strategy in [40] and aiming to alleviate information degradation during binarization, PCBNN adopts a novel contrastive learning framework to guide the binarization of activation layers.

In our setting, we treat binary and full-precision activations as discrete random variables and use mutual information (MI) to measure the shared information between them. MI captures the statistical dependence between full-precision activations and their binary counterparts, and thus reflects how much information is preserved after binarization.

For two discrete variables X and Y , the measure of MI between them can be formulated as:

$$I(X, Y) = H(X) - H(X|Y), \quad (5)$$

where $H(X)$ represents the uncertainty of X , while $H(X|Y)$ represents the uncertainty of X when Y is known. $I(X, Y)$ represents the overlapping part of information between X and Y , characterizing their correlation. If X and Y are independent, $I(X, Y)$ equals zero, which indicates no shared information. When the correlation between X and Y is stronger, the value of $I(X, Y)$ is larger as well, which means that the information in X can substitute for a significant portion of the information in Y . In this paper, the formula can be rewritten as:

$$I(A_B, A_F) = H(A_B) - H(A_B|A_F), \quad (6)$$

where A_F is full-precision activations and A_B represents binary activations. The goal is to maximize the mutual information $I(A_B, A_F)$.

The MI objective provides the theoretical motivation for preserving the information shared between binary activations and their full-precision counterparts. However, directly estimating and maximizing $I(A_B, A_F)$ is difficult because the joint distribution of A_B and A_F is unknown and dynamically changes during training. Therefore, we adopt a cosine-similarity-based contrastive loss as a tractable surrogate objective. Specifically, the binary activation and its corresponding full-precision activation are treated as a positive pair, while other activations in the same mini-batch are used as negative samples. By encouraging the positive pair to have higher similarity than negative pairs, the contrastive loss increases the statistical dependence between binary and full-precision representations, which is consistent with the objective of preserving mutual information during binarization.

Guided by this MI-preservation objective, a contrastive learning framework is introduced in the binarization task to reduce the information discrepancy between binary activations and full-precision activations. The core idea of contrastive learning is to learn meaningful feature representations by comparing the representations of two samples. As shown in Fig. 4, we replace the conventional data augmentation branch in contrastive learning with the activation binarization pathway in BNNs, where binary activations and full-precision activations are generated within the same forward process.

During the training process, binary activations and its

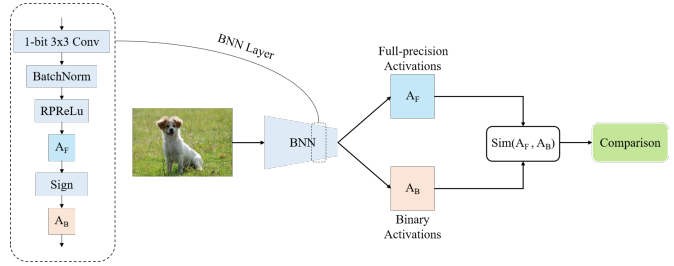


Fig. 4 Contrastive learning framework for PCBNN.

corresponding full-precision activations are used as positive pairs in the contrastive sample pairs, allowing the model to learn the similarity and dissimilarity feature representations between them. This helps the model better understand the information loss and preservation during the quantization process, thus maintaining good performance after quantization. We use cosine similarity to evaluate the similarity between two vectors. Eq. (7) shows the computation of the cosine value of the angle between the two vectors, which is denoted as $\text{Sim}(A_B, A_F)$. The cosine similarity value varies between -1 and 1, where 1 represents complete similarity and -1 signifies complete dissimilarity.

$$\text{Sim}(A_B, A_F) = \frac{A_B \cdot A_F}{\|A_B\|_2 \|A_F\|_2}. \quad (7)$$

The loss function in contrastive learning is defined by Eq. (9).

$$\mathcal{S}_i^k = \{A_B^{k,j} \mid j \neq i\} \cup \{A_F^{k,j} \mid 1 \leq j \leq N\}, \quad (8)$$

$$L_i^k = -\log \frac{\exp(\text{Sim}(A_B^{k,i}, A_F^{k,i})/\tau)}{\sum_{z \in \mathcal{S}_i^k} \exp(\text{Sim}(A_B^{k,i}, z)/\tau)}. \quad (9)$$

Here, $\mathcal{S}_i^k$ contains $2N - 1$ candidate features, including one positive feature and $2N - 2$ negative features. N represents the number of original samples in a mini-batch. For each batch, N binary representations and N full-precision representations are generated, resulting in a total of $2N$ features. For a given anchor $A_B^{k,i}$, its corresponding full-precision activation $A_F^{k,i}$ serves as the positive sample. τ represents the temperature coefficient, which is used to adjust the degree of emphasis on difficult samples. For binary network and its full-precision counterpart in the same training iteration, their activation sequences are A_B^k and A_F^k ($k \in 1, \dots, K$), respectively. We propose optimizing the loss function by maximizing the cosine similarity between sample pairs, which facilitates binary activations and full-precision activations to be closer

in the feature representation space and mitigates information degradation during quantization.

From this perspective, minimizing L_i^k does not directly compute the MI value in Eq. (6); instead, it implements the MI-preservation principle through a discriminative contrastive objective. The numerator increases the agreement between the binary activation and its corresponding full-precision activation, while the denominator suppresses the similarity to other binary or full-precision activations in the mini-batch. As a result, the learned binary activations preserve more information from their full-precision counterparts.

In summary, PCBNN constructs a contrastive learning framework for learning representative binary activations. This framework brings binary activations closer to full-precision activations while distinguishing them from other binary activations in the contrastive space.

3.3 Fully binary input layer in PCBNN

The binary input layer can effectively reduce the resource consumption of FPGA accelerators, because it no longer requires a separate design of fixed-point or floating-point convolution engines. However, we face the challenge of lacking input channels when binarizing the weights and activation values of the input layer. In this work, PCBNN introduces an innovative channel segmentation technique, and eventually realizes the fully binary input layer.

We transform the input fixed-point data into binary form using 1 and 0. As shown in Eq. (10), we take a span of 4 as a quantization interval, and round down each input value to the nearest multiple of 4, so that the single channel data image is divided into 64 binary channels. Therefore, the RGB channels are divided into 192 binary channels.

$$M = \left\lfloor \frac{X}{4} \right\rfloor. \quad (10)$$

In Eq. (10), X denotes the input data, M indicates the number of channels that are quantized to 1 after binary representation, and the number of channels that are quantized to 0 is $64 - M$.

Fig. 5 illustrates the binary representation of the input data. Fig. 5(a) shows the binarization process, where a 32×32 RGB input is transformed into $32 \times 32 \times 64 \times 3$ binary channels after channel segmentation. In Fig. 5(b), a 2×2 single-channel input is segmented into 64 binary channels, each consisting of either 0s or 1s. For example, an input value of 131 is decomposed into 32 channels of 1s and 32 channels of 0s after segmentation.

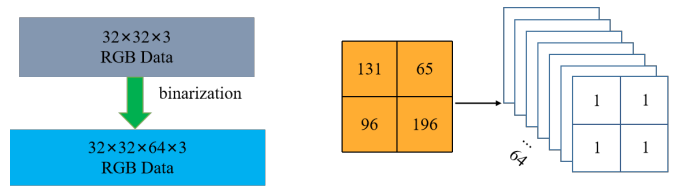


Fig. 5 Binary input data. (a) Input data binarization processing; (b) Channel division.

To maintain consistency with the subsequent representations of weights and activation values, we transform all 0s within the channels to -1s, i.e., the binary channels are converted to only contain 1s and -1s.

4 Implementation on FPGA

In this section, we design an efficient FPGA-based accelerator for the proposed PCBNN model, targeting parallel convolutions and low-power inference on resource-constrained edge and industrial platforms.

4.1 Design flow of PCBNN accelerator

This subsection presents the design flow from the PCBNN model to its FPGA-based accelerator. First, we construct a Python framework for PCBNN and apply the channel segmentation scheme described in Section 3.3, in which each input channel of the first layer is divided into 64 binary channels with a quantization step of 4, resulting in 192 binary channels for the three RGB inputs.

Next, we define the corresponding network topology for the pre-trained PCBNN model on the FPGA side and partition the computing tasks according to the layers in the forward inference stage. Similar operations are merged and each computation layer is modularized to improve development efficiency and facilitate reuse. The input data and model parameters are reorganized and quantized so that they can be efficiently fetched and cached by the target FPGA with minimal off-chip memory access.

After completing the modular design, we implement the PCBNN architecture using C++ and high-level synthesis (HLS) tools, together with manual directive tuning for loop unrolling, pipelining, and memory partitioning. Before exporting the IP core, C/RTL co-simulation is performed to verify the correctness of the design.

Finally, we integrate the generated IP core into the FPGA system. We configure system parameters in Vivado, add AXI SmartConnect, the Zynq processing system, and the HLS-generated accelerator IP. After synthesis, placement, and routing, the toolchain produces the bitstream and the corresponding hardware design of the PCBNN accelerator.

4.2 Overall architecture of PCBNN accelerator

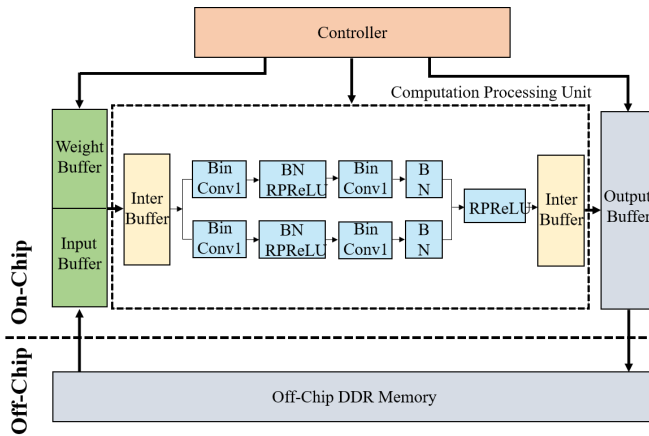


Fig. 6 PCBNN accelerator overall architecture.

As shown in Fig. 6, the overall architecture mainly consists of an on-chip PCBNN accelerator and off-chip DDR memory interconnected via an AXI bus. The PCBNN accelerator includes a controller, an input feature-map buffer, a weight buffer, an output buffer, and a computation processing unit. The computation processing unit is the core of the design. In our optimized architecture, we redesign the processing elements (PEs) and place a Bin Active module at their inputs. This allows each PE to directly process binary data streams, enabling purely binary convolutions and reducing inter-layer data movement, which is beneficial for both performance and energy efficiency on FPGAs.

The accelerator follows a hierarchical dataflow. Input images and model parameters are first transferred from off-chip DDR memory to the on-chip input feature-map buffer and weight buffer through the AXI interface, and the controller schedules the computation processing unit according to the layer order of PCBNN. During convolution, the line buffer generates sliding-window data from input feature maps, while intermediate buffers store temporary feature maps and partial sums for the PE array. Therefore, intermediate results are not immediately written back to DDR after each operation; instead, they are retained on chip whenever the following computation can consume them within the available buffer capacity.

The main data reuse takes place at three levels. Feature-map reuse is achieved by the convolution line buffer, where adjacent convolution windows share overlapping input pixels. Weight reuse is achieved by keeping binary weights in the on-chip weight buffer and reusing them across different spatial positions. Intermediate-result reuse is achieved through the output and intermediate buffers, which allow adjacent

computation modules to exchange partial results without unnecessary off-chip memory access. These mechanisms reduce redundant DDR transactions and improve effective bandwidth utilization.

For compact configurations such as CIFAR-10, most intermediate feature maps and model parameters can be retained on chip, and the network can be largely unrolled on the FPGA fabric. Thus, off-chip DDR accesses are mainly used for input loading, parameter loading when required, and final-result storage. For larger configurations such as ImageNet, feature maps are processed layer-wise or block-wise, and boundary feature maps are written back to DDR only when they exceed the available on-chip storage. This design balances on-chip data reuse and scalability for larger workloads.

The output result buffer mainly keeps the computation results generated by the current computation module. If the following layer or block can be scheduled immediately and the buffer capacity is sufficient, the results are forwarded or retained on chip for the next stage. Otherwise, the results are stored in off-chip DDR memory and reused in later processing.

4.3 Binary convolution unit

The parallel convolutional computation in PCBNN is achieved by extending a single binary neural network computing pipeline in parallel, demonstrating the potential for neural network parallel acceleration within a single chip. This architecture is particularly suitable for industrial edge devices, where high throughput and low power consumption are required under tight resource budgets.

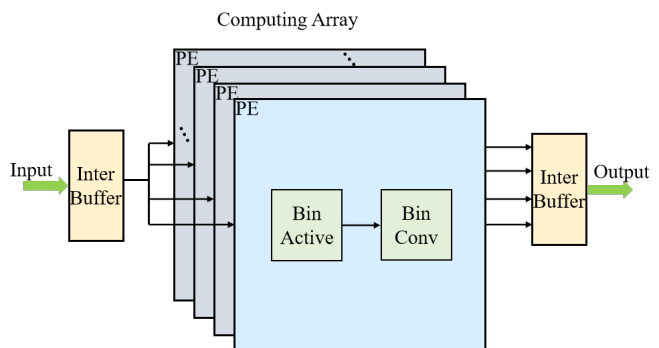


Fig. 7 Binary convolution unit.

Convolutional operations constitute a computationally intensive task, typically constituting the majority of the computational burden of the entire model. The convolutional computation unit consists of a PE array and two intermediate data buffers, as illustrated in Fig. 7. The PE is the core computing component, which consists of the Bin Active module

and the Bin Conv module. Each PE contains multiple arithmetic logic units capable of performing partial convolution computations. The convolution line buffer reads the input feature maps and generates convolution sliding window data. Then, after parallel and multiplexed computation by the Bin Active and Bin Conv modules, all the output data of the convolution channels are written into the inter buffer for the next computing array to read. The combination of the PE array completes the convolution computation for the entire layer.

4.4 Pipeline design

To enhance hardware processing performance and data throughput, we adopt a multistage pipeline for the binary convolution unit, as shown in Fig. 8. The binary convolution operation is decomposed into XNOR&Popcount, shift, and accumulation operations, which are mapped to different pipeline stages.

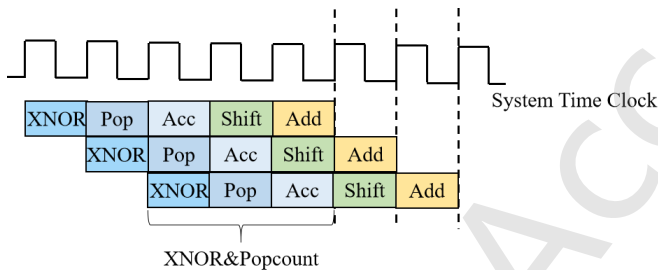


Fig. 8 Pipeline design for PCBNN.

In the proposed design, a five-stage pipeline is adopted. Three stages are dedicated to the XNOR&Popcount operations, where Popcount computes the bitwise sum of binary matching results. One stage is used for the shift operation, and the final stage accumulates partial sums across different channels or convolution windows. However, the XNOR&Popcount computation between weights and feature-map vectors typically involves long bit vectors, leading to increased critical-path latency. To address this issue, we further decompose the XNOR&Popcount computation into a three-stage sub-pipeline within the binary convolution unit.

In the first stage, bitwise XNOR is performed between the input data and the corresponding weights. In the second stage, the XNOR results are processed by a Popcount (Pop) operation to count the number of ones. In the third stage, the Popcount results from previous cycles are accumulated (Acc) to obtain the final dot-product value for each output element. The subsequent shift and accumulation stages complete the scaling and summation across partial results.

The numerical role of the shift stage is to convert the Popcount-derived matching count into the signed dot-product form used by binary convolution. For binary operands encoded by $\{-1, +1\}$, the XNOR result indicates whether the input bit and the weight bit have the same sign. If p denotes the Popcount result of the XNOR output and N denotes the length of the binary vector, the corresponding binary dot product can be computed as $2p - N$. Therefore, the shift stage implements the multiply-by-two operation in $2p - N$ through a lightweight left shift, followed by offset correction and accumulation. This avoids multiplier-based scaling and is consistent with the low-DSP design objective of the proposed FPGA accelerator.

By employing this fine-grained pipeline design, we increase the level of parallelism, reduce the critical path delay, and significantly improve the throughput of the binary convolution unit, which leads to higher overall performance of the PCBNN accelerator on FPGA-based industrial edge platforms.

5 Experiment

5.1 Datasets and experimental settings

CIFAR-10/100. The CIFAR-10 and CIFAR-100 datasets are used for image classification and consist of 60,000 32x32 color images. The CIFAR-10 dataset includes 10 categories, each with 6,000 images, while the CIFAR-100 dataset is an extended version of the CIFAR-10 dataset, encompassing 100 classes, each consisting of 600 images.

ImageNet. ImageNet is a large-scale visual recognition database with over 14 million annotated images. In this paper, we use the ImageNet ILSVRC2012 [55], which is a subset of ImageNet and widely acknowledged as one of the most extensively utilized datasets for visual recognition tasks. ImageNet ILSVRC-2012 contains 1,000 classes, comprising approximately 1.28 million training images and 50,000 validation images.

Implementation details. The proposed PCBNN is implemented in PyTorch and trained on an NVIDIA GTX 1080Ti GPU. For the CIFAR-10/100 dataset, we use random cropping techniques and horizontal flip to augment input images. We adopt a cosine scheduler [56] with a learning rate of 0.1. The optimizer used is SGD with a momentum of 0.9. During training, the binary model is trained for 250 epochs with a batch size of 256. For the ImageNet dataset, we add color jittering to augment input images. We use a cosine learning rate scheduler with an initial learning rate of 0.1. The SGD optimizer is employed with a momentum of 0.9 and a weight decay of $1e-4$. The binary model undergoes training for a total of 100 epochs, with a batch size of 256.

Then, we evaluate the performance of the accelerator based on the XCZU3EG chip of Xilinx Zynq UltraScale+ FPGA, which contains an embedded ARM CPU, 360 DSPs, 7.6 Mb BRAMs, and 71k LUTs. Finally, we assess different configurations of PCBNN, conducting synthesis using Vivado HLS and Vivado.

5.2 Network performance evaluation

In this subsection, we compare the image classification performance of the PCBNN model with the state-of-the-art (SOTA) methods.

Comparison on CIFAR-10/100 datasets. The Top-1 accuracy of PCBNN on CIFAR-10 and CIFAR-100 is shown in Table 1. Compared to the SOTA methods using the same baseline architecture ResNet-20, we achieve higher accuracy. For the CIFAR-10 dataset, we achieve 92.4% accuracy. The accuracy of PCBNN is improved by 0.5% over ISPA [44], while the accuracy disparity between BNN and full-precision model is reduced to 0.7%. For the CIFAR-100 dataset, the accuracy of PCBNN is 69.8%, which is 0.9% higher than that of A&b [43]. PCBNN narrows the accuracy disparity between BNN and its full-precision counterpart to 0.9%.

Table 1 Top-1 accuracy on CIFAR-10 (C-10) and CIFAR-100 (C-100). W/A indicates the bit width of weights and activations.

Method	Bit-width (W/A)	Acc. (%) (C-10)	Acc. (%) (C-100)
Full-precision	32/32	93.1	70.7
RBNN [39]	1/1	87.0	66.0
CMIM [40]	1/1	87.6	68.2
AdaBin [41]	1/1	88.2	-
BiPer [42]	1/1	91.2	68.6
A&b [43]	1/1	91.7	68.9
ISPA [44]	1/1	91.9	69.4
PCBNN(Ours)	1/1	92.4	69.8

Comparison on ImageNet dataset. Table 2 presents a comparison of Top-1 and Top-5 accuracy metrics for ImageNet. To ensure comparability with SOTA method [39–44], we adopt ResNet-18 as the baseline architecture. The PCBNN achieves improvements in both Top-1 and Top-5. Especially in terms of Top-1 accuracy, PCBNN achieves 63.2% accuracy, which is only 6.4% accuracy difference compared to its full-precision counterpart. This result further validates the stability of our proposed PCBNN on large-scale datasets.

Table 2 Top-1 and Top-5 accuracy on ImageNet.

Method	Bit-width (W/A)	Top-1 (%)	Top-5 (%)
Full-precision	32/32	69.6	89.2
RBNN [39]	1/1	59.9	81.9
CMIM [40]	1/1	62.5	84.2
AdaBin [41]	1/1	63.1	84.3
BiPer [42]	1/1	62.3	84.4
A&b [43]	1/1	62.4	84.7
ISPA [44]	1/1	62.8	85.0
PCBNN(Ours)	1/1	63.2	85.3

5.3 Ablation study

In this part, we conduct ablation experiments on the proposed parallel convolutional architecture, contrastive learning framework, and binary input layer to validate the efficacy of the proposed PCBNN.

To clearly evaluate the contribution of each proposed component, we define the ablation variants as follows. The baseline denotes the original binary ResNet architecture without the proposed modules. Model A denotes the baseline equipped with the proposed parallel convolutional architecture. Model A+B further incorporates the contrastive learning framework on top of Model A. The final PCBNN additionally adopts the proposed fully binarized input layer based on channel segmentation. Based on these variants, we evaluate the effectiveness of the parallel convolutional architecture, the contrastive learning framework, and the binary input layer, respectively.

Parallel convolutional architecture. ResNet is adopted as the baseline network. We validate the effectiveness of our proposed parallel convolutional architecture (Model A) on the CIFAR-10/CIFAR-100 and ImageNet datasets, as shown in Table 3. Compared with the baseline, Model A improves the accuracy by 7.8% and 5% for CIFAR-10 and ImageNet, respectively. The experimental results highlight the significant performance improvement of the proposed parallel convolutional architecture.

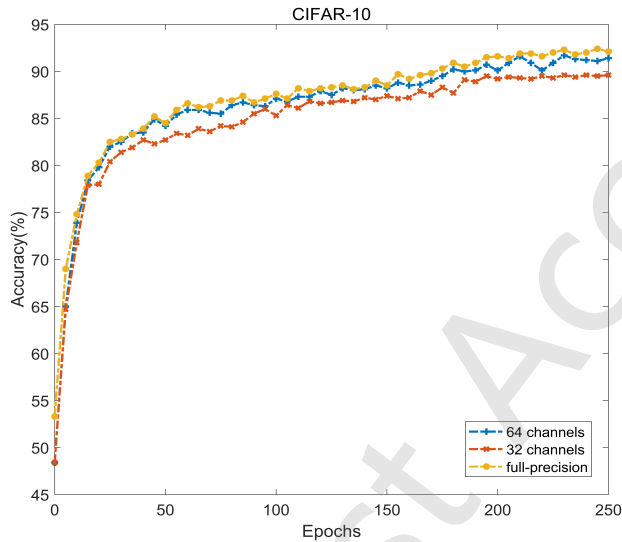
Contrastive learning framework. Based on the proposed parallel convolutional architecture, we further verify the impact of introducing the contrastive learning framework (Model A+B) on model accuracy. The results are shown in Table 4. From Table 4, it can be seen that the contrastive learning framework can significantly improve the recognition accuracy, which illustrates the effectiveness of our proposed method.

Table 3 The impact of parallel convolutional architecture on accuracy in PCBNN.

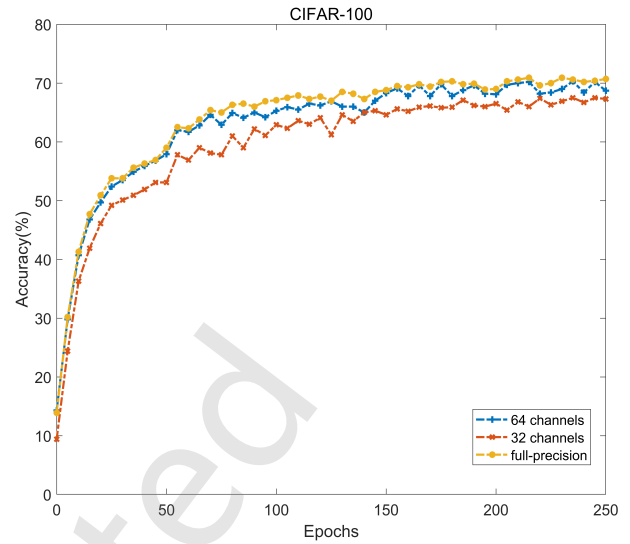
	Baseline network	Proposed parallel convolutional architecture
CIFAR-10	80.1	87.9
CIFAR-100	59.4	65.8
ImageNet	54.7	59.7

Table 4 Performance comparison with and without contrastive learning.

	Non-Contrastive Learning	Contrastive Learning
CIFAR-10	87.9	91.1
CIFAR-100	65.8	69.0
ImageNet	59.7	62.5

**Fig. 9** Performance comparison of different input channels in CIFAR-10.

Binary input layer. Lastly, based on Model A+B, we further investigate the effect of the number of input channels in the first layer on accuracy. As shown in Fig. 9, the results show that our proposed method of dividing a single input channel into 64 channels using a stride of 4 outperforms the method of dividing a single input channel into 32 channels using thermometer encoding [49]. This improved channel division method leads to a significant enhancement in recognition accuracy, achieving 92.4% on the CIFAR-10 dataset, reducing the gap compared to the first layer using full-precision fixed-point quantization to just 0.7% and outperforming [49] by 1.6%. In addition, as indicated in Fig. 10, on the CIFAR-100 dataset, our binary input layer method with a span of 4 reduced the gap to 1.8% compared to the input layer using

**Fig. 10** Performance comparison of different input channels in CIFAR-100.

full-precision fixed-point quantization, surpassing [49] by 0.8%.

Table 5 Hardware performance comparison on CIFAR-10.

	FP-BNN [32]	FBNA [48]	FracBNN [49]	Zhang <i>et al.</i> [52]	PCBNN
Device	Stratix-V 5SGSD8	Zynq ZC702	Zynq ZU3EG	Zynq ZU3EG	Zynq ZU3EG
Bits(W/A)	2/8	1/1	1/1.4	1/1	1/1
Fmax(MHz)	150	-	250	220	250
LUT	-	29600	50656	61274	54511
DSP	20	-	224	217	143
BRAM	2210	103	201	200	205.5
FPS	-	520.8	2806.9	1521.0	1639.3
Power(W)	26.2	3.3	4.1	4.5	2.9
FPS/W	-	157.82	684.61	338.00	565.28

5.4 Hardware evaluation

In this subsection, we evaluate the FPGA implementation of the PCBNN accelerator.

5.4.1 Hardware performance comparison

It should be noted that the compared accelerators differ in FPGA devices, numerical precision settings, and implementation strategies. For example, the compared designs use different bit-width configurations, including 2/8, 4/4, 1/1.4, and 1/1. Therefore, the comparison should be interpreted as a

system-level reference rather than a strictly identical-platform comparison. To provide a more informative evaluation of energy efficiency under these differences, we additionally report FPS/W in Tables 5 and 6.

Table 5 shows the hardware performance comparison on CIFAR-10. Compared to FracBNN [49] and FBNA [48], our design achieves faster frame rates and lower power consumption on similar FPGA platforms. Due to the compact nature of the PCBNN model on CIFAR-10, we can unroll the entire network on the FPGA fabric, eliminating redundant transactions between DDR memory and logic blocks. The only data transfers are the input image and the prediction results. This deployment style allows most intermediate features and model parameters to remain on chip, which effectively alleviates external memory bandwidth pressure and reduces unnecessary energy overhead. With this setup, our design excels in terms of power consumption, consuming only 2.9 watts of power. Compared with FracBNN [49], we successfully save 29.3% power consumption, which is more suitable for deployment on resource-constrained mobile edge devices. In terms of normalized energy efficiency, PCBNN achieves 565.28 FPS/W on CIFAR-10. Although FracBNN reports a higher FPS/W on this compact workload, PCBNN achieves the lowest absolute power consumption and maintains a fully binarized 1/1 computation pipeline without fractional-activation processing.

Table 6 Hardware performance comparison on ImageNet.

	FP-BNN	JPEGComp	FracBNN	Zhang <i>et al.</i>	PCBNN
	[32]	[51]	[49]	[52]	
Device	Stratix-V 5SGSD8	Zynq ZU3EG	Zynq ZU3EG	Zynq ZU3EG	Zynq ZU3EG
Bits(W/A)	2/8	4/4	1/1.4	1/1	1/1
Fmax(MHz)	150	250	250	220	250
LUT	-	51776	51444	63000	53126
DSP	384	360	126	130.1	231
BRAM	2210	159	212	202	216
FPS	862.1	41.1	48.1	129.1	194.9
Power(W)	26.2	5.5	6.1	7.1	4.4
FPS/W	32.90	7.47	7.89	18.18	44.30

Table 6 presents the comparison between the proposed PCBNN accelerator and state-of-the-art FPGA-based BNN accelerators on the ImageNet dataset. As shown in the table, PCBNN demonstrates clear advantages in both resource efficiency and runtime performance. Compared with prior

BNN accelerators in [32] and [51], the proposed design significantly reduces hardware overhead. Specifically, the power consumption and BRAM usage of PCBNN are reduced by $5.9\times$ and $10.2\times$, respectively. This improvement mainly benefits from the fully binarized computation pipeline and the elimination of redundant data movement, which together reduce both memory footprint and energy consumption.

Furthermore, when compared to recent FPGA implementations with similar device scales [49, 52], PCBNN achieves the highest frame rate with comparable resource utilization, indicating that the performance gain mainly comes from architectural parallelism and dataflow optimization rather than from larger devices or higher operating frequencies. In particular, our design reaches 194.9 frames per second (FPS), which is approximately four times faster than FracBNN [49] under the same hardware constraints, while also consuming about one-third less power. Moreover, PCBNN achieves 44.30 FPS/W on ImageNet, which is higher than the compared FPGA accelerators and demonstrates its advantage in normalized energy efficiency for large-scale workloads. This indicates that the proposed architecture not only improves throughput but also enhances energy efficiency simultaneously.

Table 7 FPGA resource utilization of PCBNN on different datasets.

	LUT	BRAM	DSP
CIFAR-10	54511(77.25%)	205.5(95.14%)	143(39.72%)
CIFAR-100	51878(73.52%)	216(100%)	134(37.22%)
ImageNet	53126(75.29%)	216(100%)	231(64.17%)
Available	70560	216	360

5.4.2 Hardware resource analysis

After synthesis and implementation in Vivado, the detailed resource utilization of the design is summarized in Table 7. We can find that the utilization of DSP resources is relatively low. Specifically, on smaller datasets such as CIFAR-10/CIFAR-100, the DSP utilization stays below 40%, while for large-scale datasets like ImageNet, it does not exceed 65%. This result is consistent with our design philosophy, where conventional floating-point multiplications are replaced by XNOR and Popcount operations. As a result, the dependence on DSP blocks is significantly reduced, allowing the design to rely more on lightweight logic resources.

In contrast, the utilization of LUT resources is at a moderate level, reflecting the mapping of bitwise operations to FPGA logic fabric. Since XNOR and Popcount operations can be efficiently implemented using LUTs, the design achieves a

Table 8 Hardware platform performance comparison.

Dataset	CPU			GPU			FPGA		
	Time (ms)	Power (Watt)	imgs/sec/Watt	Time (ms)	Power (Watt)	imgs/sec/Watt	Time (ms)	Power (Watt)	imgs/sec/Watt
CIFAR-10	24.80	65	0.62	0.84	250	4.76	0.61	2.93	559.50
ImageNet	499.78	65	0.03	25.67	250	0.16	5.13	4.44	43.90

balanced use of logic resources without introducing excessive hardware overhead. Furthermore, the utilization of BRAM is relatively high, which aligns with the memory-intensive nature of convolutional neural networks. The need to store intermediate feature maps, weights, and buffering data contributes to this trend. Overall, the resource distribution demonstrates that the proposed PCBNN accelerator effectively shifts computation from DSP-dominated arithmetic units to logic and memory resources, achieving a more hardware-friendly and efficient deployment on FPGA platforms.

For CIFAR-100 and ImageNet, the BRAM utilization reaches 216/216, indicating that the available on-chip memory resources are fully used. This high BRAM utilization mainly comes from the storage of intermediate feature maps, binary weights, line buffers, and inter-stage buffers required by the binary convolution pipeline. In practical implementation, high BRAM utilization may increase the difficulty of routing and timing closure. To alleviate these issues, we organize the on-chip buffers in a modular manner and adopt buffer banking, memory partitioning, and pipeline-level scheduling to reduce memory access conflicts and avoid concentrating all memory accesses on a single timing-critical path. After these optimizations, the implemented design successfully meets the target operating frequency reported in Tables 5 and 6. Nevertheless, the high BRAM utilization indicates that on-chip memory remains a critical resource for scaling PCBNN to larger models. In future work, we will investigate more flexible buffer compression and tiling strategies to further improve implementation scalability.

5.4.3 Comparison on different hardware platforms

Table 8 reports the hardware performance comparison among CPU, GPU, and the FPGA-based PCBNN accelerator using a Core i7-11700 CPU, an NVIDIA GTX 1080Ti GPU, and an XCZU3EG FPGA, respectively. Overall, the proposed PCBNN accelerator demonstrates clear advantages in both latency and energy efficiency. In terms of inference latency, the FPGA implementation consistently outperforms conventional platforms, achieving over one order-of-magnitude speedup compared with the CPU and noticeable gains over the GPU. This advantage becomes even more pronounced

on the more complex ImageNet workload, demonstrating effective scalability with increasing computational intensity.

This advantage can be attributed to the algorithm–hardware co-design of PCBNN. By reformulating convolution into XNOR and Popcount operations, the accelerator maps computation to lightweight bitwise logic and fully exploits fine-grained parallelism on FPGA fabric. Such a design not only shortens the critical path but also improves pipeline efficiency and resource utilization, enabling sustained high-throughput execution. From the energy perspective, the FPGA-based accelerator exhibits significantly lower power consumption than both CPU and GPU platforms, while delivering substantially higher throughput per watt. This highlights a key benefit of customized hardware acceleration, where computation patterns are tightly aligned with hardware characteristics.

6 Conclusion

This paper presented PCBNN, an algorithm–hardware co-designed binary neural network for low-power edge computing systems. By integrating a parallel convolutional architecture, a contrastive-learning-guided optimization strategy, and a channel-segmentation-based fully binarized input layer, the proposed design improves the representation capability of binary networks while maintaining hardware efficiency. An FPGA-based accelerator was further developed using high-level synthesis to support efficient deployment. Experimental results on CIFAR-10, CIFAR-100, and ImageNet show that PCBNN achieves competitive accuracy with favorable improvements in energy efficiency compared with representative BNN approaches. The results also demonstrate that the proposed co-design effectively exploits FPGA parallelism and on-chip resources, enabling low-latency inference under constrained hardware budgets. Overall, this work highlights the importance of algorithm–hardware co-design for bridging binary neural network optimization and practical deployment. Future work will focus on more scalable input encoding strategies and extending the proposed framework to broader edge platforms and applications.

Reference

- [1] S. Li, T. Cui, and W. Viriyasitavat, Edge device fault probability based intelligent calculations for fault probability of smart

- systems, *Tsinghua Sci. Technol.*, vol. 29, no. 4, pp. 1023–1036, 2024.
- [2] Y. Guo, W. Xie, Q. Wang, D. Yan, and Y. Zhang, Betweenness approximation for edge computing with hypergraph neural networks, *Tsinghua Sci. Technol.*, vol. 30, no. 1, pp. 331–344, 2024.
 - [3] T. Wang, P. Wang, S. Cai, X. Zheng, Y. Ma, W. Jia, and G. Wang, Mobile edge-enabled trust evaluation for the Internet of Things, *Inf. Fusion*, vol. 75, pp. 90–100, 2021.
 - [4] Y. Liang, M. Yin, W. Wang, Q. Liu, L. Wang, X. Zheng, and T. Wang, Collaborative edge server placement for maximizing QoS with distributed data cleaning, *IEEE Trans. Serv. Comput.*, vol. 18, no. 3, pp. 1321–1335, 2025.
 - [5] R. Chen, C. Yi, H. Zhu, W. Wu, J. Kang, and D. Niyato, Dynamic digital twin update by adaptive model splitting and reliable crowdsourcing under uncertain data distortions, *IEEE Trans. Mob. Comput.*, pp. 1–16, 2026.
 - [6] R. Chen, C. Yi, F. Zhou, J. Kang, Y. Wu, and D. Niyato, Federated digital twin construction via distributed sensing: A game-theoretic online optimization with overlapping coalitions, *IEEE Trans. Mob. Comput.*, vol. 24, no. 11, pp. 12221–12238, 2025.
 - [7] R. Chen and C. Yi, A game-theoretic online optimization for federated digital twin construction via wireless sensing, in *Proc. IEEE Int. Conf. Commun. (ICC)*, 2025, pp. 765–770.
 - [8] L. Hu, A. Liu, M. Xie, and T. Wang, UAVs joint vehicles as data mules for fast codes dissemination for edge networking in smart city, *Peer-to-Peer Netw. Appl.*, vol. 12, no. 6, pp. 1550–1574, 2019.
 - [9] X. Liu, T. Qiu, and T. Wang, Load-balanced data dissemination for wireless sensor networks: A nature-inspired approach, *IEEE Internet Things J.*, vol. 6, no. 6, pp. 9256–9265, 2019.
 - [10] Y. Liang, M. Yin, Y. Zhang, W. Wang, W. Jia, and T. Wang, Grouping reduces energy cost in directionally rechargeable wireless vehicular and sensor networks, *IEEE Trans. Veh. Technol.*, vol. 72, no. 8, pp. 10840–10851, 2023.
 - [11] C. Xu, J. Guo, Y. Liang, H. Zou, J. Zeng, H. Dai, W. Jia, J. Cao, and T. Wang, Enhancing QoE in collaborative edge systems with feedback diffusion generative scheduling, *IEEE Trans. Mob. Comput.*, vol. 24, no. 12, pp. 12983–12998, 2025.
 - [12] M. K. Bhuyan, M. Kamruzzaman, S. I. Nilima, R. Khatoun, and N. Mohammad, Convolutional neural networks based detection system for cyber-attacks in industrial control systems, *J. Comput. Sci. Technol. Stud.*, vol. 6, no. 3, pp. 86–96, 2024.
 - [13] Y. Mei, W. Wang, Y. Liang, Q. Liu, S. Chen, and T. Wang, Privacy-enhanced cooperative storage scheme for contact-free sensory data in AIoT with efficient synchronization, *ACM Trans. Sensor Netw.*, vol. 20, no. 4, Art. no. 84, pp. 1–19, 2024.
 - [14] T. Wang, Y. Liang, X. Shen, X. Zheng, A. Mahmood, and Q. Z. Sheng, Edge computing and sensor-cloud: Overview, solutions, and directions, *ACM Comput. Surv.*, vol. 55, no. 13s, pp. 1–37, 2023.
 - [15] Z. Peng, N. Zou, J. Zeng, B. Li, S. Chen, T. Wang, and W. Jia, Dynamic grouping and aggregation weight optimization for hierarchical federated learning with quantization, *IEEE/ACM Trans. Netw.*, vol. 34, pp. 3824–3839, 2026.
 - [16] X. Zhao, L. Wang, Y. Zhang, X. Han, M. Deveci, and M. Parmar, A review of convolutional neural networks in computer vision, *Artif. Intell. Rev.*, vol. 57, no. 4, p. 99, 2024.
 - [17] C. Zhang, B. Lin, X. Hu, S. Qi, L. Qian, and Y. Wu, Resource management and trajectory optimization for UAV-IRS assisted maritime edge computing networks, *Tsinghua Sci. Technol.*, vol. 30, no. 4, pp. 1600–1616, 2025.
 - [18] X. Liu, J. Lv, B.-G. Kim, K. Li, H. Jin, W. Gao, and J. Bai, Cooperative digital healthcare task scheduling and resource management in edge intelligence systems, *Tsinghua Sci. Technol.*, vol. 30, no. 2, pp. 926–945, 2024.
 - [19] R. Chen, C. Yi, K. Zhu, B. Chen, J. Cai, and M. Guizani, A three-party hierarchical game for physical layer security aware wireless communications with dynamic trilateral coalitions, *IEEE Trans. Wireless Commun.*, vol. 23, no. 5, pp. 4815–4829, 2024.
 - [20] D. Ren, W. Li, J. Huo, *et al.*, “Leveraging frequency analysis for image denoising network pruning,” *IEEE Trans. Image Process.*, 2025.
 - [21] E. Shang, H. Liu, J. Zhang, R. Zhao, and J. Du, Ensemble knowledge distillation for federated semi-supervised image classification, *Tsinghua Sci. Technol.*, vol. 30, no. 1, pp. 112–123, 2024.
 - [22] M. Fu, H. Yu, J. Shao, *et al.*, “Quantization without tears,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2025, pp. 4462–4472.
 - [23] J. Hou, X. Chen, R. Ran, *et al.*, “Binarized neural network for multi-spectral image fusion,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2025, pp. 2236–2245.
 - [24] M. Courbariaux, I. Hubara, D. Soudry, R. El-Yaniv, and Y. Bengio, “Binarized neural networks: Training deep neural networks with weights and activations constrained to +1 or -1,” *arXiv preprint arXiv:1602.02830*, 2016.
 - [25] X. Jia, B. Zhang, Z. Wu, J. Yang, X. Wang, and W. Zhao, High-performance Bayesian neural network inference accelerator based on FPGA, *IEEE Trans. Circuits Syst. Artif. Intell.*, 2025.
 - [26] A. Boutros, A. Arora, and V. Betz, Field-programmable gate array architecture for deep learning: Survey and future directions, *Proc. IEEE*, 2025.
 - [27] Y. Zhang, J. Zheng, H. Meng, C. Zhong, and G. Chen, eTSDF: An energy-efficient accelerator for TSDF-based 3D scene reconstruction in FPGAs, *Tsinghua Sci. Technol.*, vol. 31, no. 2, pp. 693–707, 2026. (in Chinese)
 - [28] F. Huang, W. Wang, Q. Liu, W. Fan, W. Su, W. Jia, T. Wang, and J. Cao, EdgeManager: Online adaptive resource management for hierarchical DNN inference in collaborative edge environments, *IEEE Trans. Mob. Comput.*, vol. 25, no. 4, pp. 5571–5589, 2026.
 - [29] Y. Ling, T. He, H. Meng, Y. Zhang, and G. Chen, “Hardware accelerator for an accurate local stereo matching algorithm

- using binary neural network,” *J. Syst. Archit.*, vol. 117, p. 102110, 2021.
- [30] E. Wang, J. J. Davis, P. Y. K. Cheung, and G. A. Constantinides, LUTNet: Learning FPGA configurations for highly efficient neural network inference, *IEEE Trans. Comput.*, vol. 69, no. 12, pp. 1795–1808, 2020.
- [31] S. Aoki, S. Koyama, and T. Saito, “Theoretical analysis of dynamic binary neural networks with simple sparse connection,” *Neurocomputing*, vol. 341, pp. 149–155, 2019.
- [32] S. Liang, S. Yin, L. Liu, W. Luk, and S. Wei, “FP-BNN: Binarized neural network on FPGA,” *Neurocomputing*, vol. 275, pp. 1072–1086, 2018.
- [33] H. Nakahara, T. Fujii, and S. Sato, A fully connected layer elimination for a binarized convolutional neural network on an FPGA, in *Proc. IEEE Int. Conf. Field-Program. Logic Appl. (FPL)*, 2017, pp. 1–4.
- [34] R. Zhao, W. Song, W. Zhang, T. Xing, J.-H. Lin, M. Srivastava, R. Gupta, and Z. Zhang, Accelerating binarized convolutional neural networks with software-programmable FPGAs, in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays (FPGA)*, 2017, pp. 15–24.
- [35] G. Yang, J. Lei, Z. Fang, Y. Li, J. Zhang, and W. Xie, Hybnn: Quantifying and optimizing hardware efficiency of binary neural networks, *ACM Trans. Reconfigurable Technol. Syst.*, vol. 17, no. 2, pp. 1–24, 2024.
- [36] K. Zeng, Z. Wan, H. W. Gu, and T. Shen, “Self-knowledge distillation enhanced binary neural networks derived from underutilized information,” *Applied Intelligence*, vol. 54, no. 6, pp. 4994–5014, 2024.
- [37] Z. Liu, Z. Shen, M. Savvides, and K.-T. Cheng, “ReactNet: Towards precise binary neural network with generalized activation functions,” in *Proc. Eur. Conf. Comput. Vis.*, pp. 143–159, 2020.
- [38] Z. Wang, J. Lu, C. Tao, J. Zhou, and Q. Tian, “Learning channel-wise interactions for binary convolutional neural networks,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, pp. 568–577, 2019.
- [39] M. Lin, R. Ji, Z. Xu, B. Zhang, Y. Wang, Y. Wu, F. Huang, and C.-W. Lin, “Rotated binary neural network,” *Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 7474–7485, 2020.
- [40] Y. Shang, D. Xu, Z. Zong, L. Nie, and Y. Yan, “Network binarization via contrastive learning,” in *Proc. Eur. Conf. Comput. Vis.*, pp. 586–602, 2022.
- [41] Z. Tu, X. Chen, P. Ren, and Y. Wang, “Adabin: Improving binary neural networks with adaptive binary sets,” in *Proc. Eur. Conf. Comput. Vis.*, pp. 379–395, 2022.
- [42] E. Vargas, C. V. Correa, C. Hinojosa, and H. Arguello, BiPer: Binary neural networks using a periodic function, in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 5684–5693.
- [43] R. Ma, G. Qiao, Y. Liu, L. Meng, N. Ning, Y. Liu, and S. Hu, A&B BNN: Add&bit-operation-only hardware-friendly binary neural network, in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 5704–5713.
- [44] L. Orecchia, J. Hu, X. He, Z. Wang, X. Yang, M. Wu, and X. Geng, Training binary neural networks via Gaussian variational inference and low-rank semidefinite programming, *Adv. Neural Inf. Process. Syst.*, vol. 37, pp. 63963–63985, 2024.
- [45] H. Nakahara, H. Yonekawa, T. Fujii, and S. Sato, A lightweight YOLOv2: A binarized CNN with a parallel support vector regression for an FPGA, in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays (FPGA)*, 2018, pp. 31–40.
- [46] E. Nurvitadhi, D. Sheffield, J. Sim, A. Mishra, G. Venkatesh, and D. Marr, Accelerating binarized neural networks: Comparison of FPGA, CPU, GPU, and ASIC, in *Proc. IEEE Int. Conf. Field-Program. Technol. (FPT)*, 2016, pp. 77–84.
- [47] T. Hirtzlin, B. Penkovsky, M. Bocquet, J.-O. Klein, J.-M. Portal, and D. Querlioz, “Stochastic computing for hardware implementation of binarized neural networks,” *IEEE Access*, vol. 7, pp. 76394–76403, 2019.
- [48] P. Guo, H. Ma, R. Chen, P. Li, S. Xie, and D. Wang, “FBNA: A fully binarized neural network accelerator,” in *Proc. 28th Int. Conf. Field Programmable Logic and Applications*, pp. 51–513, 2018.
- [49] Y. Zhang, J. Pan, X. Liu, H. Chen, D. Chen, and Z. Zhang, FracBNN: Accurate and FPGA-efficient binary neural networks with fractional activations, in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays (FPGA)*, 2021, pp. 171–182.
- [50] M. Ghasemzadeh, M. Samragh, and F. Koushanfar, ReB-Net: Residual binarized neural network, in *Proc. IEEE Int. Symp. Field-Program. Custom Comput. Mach. (FCCM)*, 2018, pp. 57–64.
- [51] H. Nakahara, Z. Que, and W. Luk, High-throughput convolutional neural network on an FPGA by customized JPEG compression, in *Proc. IEEE Int. Symp. Field-Program. Custom Comput. Mach. (FCCM)*, 2020, pp. 1–9.
- [52] D. Zhang, R. Cen, H. Pu, R. Wan, and D. Wang, An FPGA-based binary neural network accelerator with enhanced hardware efficiency and data reuse, *Microelectron. J.*, vol. 156, Art. no. 106556, 2025.
- [53] M. Lin, R. Ji, Z. Xu, B. Zhang, F. Chao, C.-W. Lin, and L. Shao, Siman: Sign-to-magnitude network binarization, *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 5, pp. 6277–6288, 2022.
- [54] P. Yin, J. Lyu, S. Zhang, S. Osher, Y. Qi, and J. Xin, “Understanding straight-through estimator in training activation quantized neural nets,” *arXiv preprint arXiv:1903.05662*, 2019.
- [55] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “ImageNet: A large-scale hierarchical image database,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, pp. 248–255, 2009.
- [56] J. Vahidi and S. M. Golmaei, Optimizing construction scheduling using the sine cosine algorithm (SCA) with building information modeling (BIM), *Results Eng.*, vol. 28, Art. no. 107100, 2025.